EXPERIMENT – 11

Title

To Understand Working with PL/SQL

Aim / Objective

To understand the basics of **PL/SQL (Procedural Language/Structured Query Language)** by writing and executing simple PL/SQL programs using variables, operators, input/output statements, conditional statements, and loops.

Theory

PL/SQL (Procedural Language/Structured Query Language) is Oracle's procedural extension to SQL. It combines SQL statements with procedural programming constructs such as variables, loops, conditional statements, exception handling, procedures, functions, and triggers.

Unlike SQL, which is declarative, PL/SQL allows programmers to write complete programs that execute business logic inside the Oracle Database.

PL/SQL programs are executed in the Oracle Database Engine and provide better performance by reducing communication between the application and the database.

Features of PL/SQL

- Block-structured language
- Supports variables and constants
- Provides conditional statements
- Supports loops
- Handles exceptions
- Integrates SQL and procedural programming
- Improves application performance

Structure of a PL/SQL Block

A PL/SQL block consists of three sections:

1. **Declaration Section (Optional)** – Variables, constants, and cursors are declared.
2. **Execution Section (Mandatory)** – Contains executable statements.

3. **Exception Section (Optional)** – Handles runtime errors.

```
DECLARE
  -- Variable Declaration
BEGIN
  -- Executable Statements
EXCEPTION
  -- Exception Handling
END;
/
```

Learning Objectives

After completing this experiment, students will be able to:

- Understand the structure of a PL/SQL block.
- Declare and use variables.
- Display output using DBMS_OUTPUT.
- Perform arithmetic operations.
- Use conditional statements.
- Implement loops.
- Execute PL/SQL programs successfully.

Software Requirements

- Oracle Database
- Oracle SQL Developer / SQL*Plus

Algorithm / Procedure

1. Open Oracle SQL Developer.
2. Connect to the Oracle database.
3. Enable DBMS Output.
4. Write PL/SQL programs.
5. Execute the programs.
6. Observe the output.
7. Stop.

Program 1 : Simple PL/SQL Block

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL Programming');
END;
/
```

Expected Output

Welcome to PL/SQL Programming

PL/SQL procedure successfully completed.

Program 2 : Variable Declaration

```
DECLARE
  Name VARCHAR2(30) := 'Rahul';
BEGIN
  DBMS_OUTPUT.PUT_LINE('Student Name : ' || Name);
END;
/
```

Expected Output

Student Name : Rahul

Program 3 : Addition of Two Numbers

```
DECLARE
  A NUMBER := 20;
  B NUMBER := 30;
  SUM1 NUMBER;
BEGIN
  SUM1 := A + B;

  DBMS_OUTPUT.PUT_LINE('Sum = ' || SUM1);
END;
/
```

Expected Output

Sum = 50

Program 4 : IF Statement

```
DECLARE
  Marks NUMBER := 75;
BEGIN
```

```
   IF Marks>=40 THEN

      DBMS_OUTPUT.PUT_LINE('PASS');

   END IF;

END;
/
```

Expected Output

PASS

Program 5 : IF-ELSE Statement

```
DECLARE
   Age NUMBER := 17;
BEGIN

   IF Age>=18 THEN

      DBMS_OUTPUT.PUT_LINE('Eligible to Vote');

   ELSE

      DBMS_OUTPUT.PUT_LINE('Not Eligible');

   END IF;

END;
/
```

Expected Output

Not Eligible

Program 6 : FOR Loop

```
BEGIN

   FOR I IN 1..5 LOOP

      DBMS_OUTPUT.PUT_LINE(I);

   END LOOP;

END;
/
```

Expected Output

1
2
3
4
5

Program 7 : WHILE Loop

```
DECLARE
   I NUMBER:=1;
BEGIN

   WHILE I<=5 LOOP

      DBMS_OUTPUT.PUT_LINE(I);

      I:=I+1;

   END LOOP;

END;
/
```

Expected Output

1
2
3
4
5

Program 8 : Display Employee Details

```
DECLARE

   V_Name VARCHAR2(30):='Rahul';

   V_Salary NUMBER:=35000;

BEGIN

   DBMS_OUTPUT.PUT_LINE('Employee  Name : '||V_Name);

   DBMS_OUTPUT.PUT_LINE('Salary  : '||V_Salary);

END;
/
```

Expected Output

Employee Name : Rahul

Salary : 35000

Flowchart

```
Start
  |
  ▼
Declare Variables
  |
  ▼
Begin Execution
  |
  ▼
Execute Statements
  |
  ▼
Display Output
  |
  ▼
Exception (if any)
  |
  ▼
End
```

Explanation

DECLARE

Used to declare variables and constants.

BEGIN

Marks the beginning of executable statements.

DBMS_OUTPUT.PUT_LINE

Displays output on the screen.

END

Marks the end of the PL/SQL block.

/

Executes the PL/SQL block in SQL*Plus or SQL Developer.

Observation

- PL/SQL blocks executed successfully.
- Variables stored and displayed values correctly.
- Arithmetic operations produced the expected results.
- Conditional statements worked correctly.
- Loops executed repeatedly as expected.

Result

Basic PL/SQL programs were successfully executed. Variables, arithmetic operations, conditional statements, loops, and output statements were implemented successfully, demonstrating the working of PL/SQL blocks.

Applications

- Banking Systems
- Payroll Applications
- Student Information Systems
- Hospital Management Systems
- Inventory Management Systems
- Railway Reservation Systems
- Enterprise Database Applications

Advantages

- Supports procedural programming.
- Reduces network traffic.
- Improves execution speed.
- Provides exception handling.
- Supports modular programming.

- Integrates SQL with programming logic.

Viva Questions

1. What is PL/SQL?
2. What is the difference between SQL and PL/SQL?
3. What are the sections of a PL/SQL block?
4. Which section of a PL/SQL block is mandatory?
5. What is the purpose of the DECLARE section?
6. What is DBMS_OUTPUT.PUT_LINE?
7. What is the purpose of the slash (/) after a PL/SQL block?
8. What are the advantages of PL/SQL?
9. Can SQL statements be used inside PL/SQL?
10. Why is PL/SQL called a procedural language?

Lab Assignment

1. Write a PL/SQL program to calculate the area of a rectangle.
2. Write a PL/SQL program to check whether a number is even or odd.
3. Write a PL/SQL program to find the largest of two numbers.
4. Write a PL/SQL program to print numbers from 1 to 20 using a FOR loop.
5. Write a PL/SQL program to calculate the factorial of a number.
6. Write a PL/SQL program to display employee details using variables.
7. Write a PL/SQL program to calculate the simple interest.
8. Compare SQL and PL/SQL with suitable examples.
9. Explain the structure of a PL/SQL block.
10. Write a PL/SQL program using an IF-ELSE statement.

Precautions

- Enable **DBMS Output** before executing PL/SQL programs in Oracle SQL Developer.

- End every PL/SQL block with END; followed by /.
- Declare variables with appropriate data types.
- Use proper syntax for loops and conditional statements.
- Check for compilation errors before execution.

Learning Outcome

After completing this experiment, students will be able to:

- Understand the architecture and working of PL/SQL.
- Write and execute anonymous PL/SQL blocks.
- Declare and use variables effectively.
- Implement arithmetic operations, conditional statements, and loops.
- Display output using DBMS_OUTPUT.PUT_LINE.
- Develop simple PL/SQL programs as a foundation for advanced concepts such as procedures, functions, cursors, and triggers.